

TA Recruitment System Group Report

Group Number: 65

Date: May 23, 2026

Project Sponsor: BUPT International School

Team Members:

Member	QM Number	Name
1	231226668	Yutong Zhou
2	231226716	Hanxin Lyu
3	231226794	Yixuan Zhang
4	231226749	Hongbo Wang
5	231226819	Xingyu Liang
6	231226543	Tie Wang

1. Introduction

This report documents the software design strategy, build plan, testing strategies, and the use of Generative AI in the development of the Teaching Assistant (TA) Recruitment System for BUPT International School. The system addresses critical pain points identified in the current manual recruitment process, including scattered applicant information, unclear application status tracking, and inefficient workload management.

2. Software Design Strategy

2.1 Background and Requirements Analysis

Problem Statement:

The current TA recruitment process relies heavily on manual forms and Excel files, creating the following issues:

Scattered applicant information across multiple documents

Unclear application status for candidates

Inefficient workload management across courses

Timeconsuming manual applicant comparison

Requirements Elicitation Methods:

The project team employed multiple techniques to gather comprehensive requirements:

Method	Participants	Key Findings
Semistructured Interviews	2 Module Organisers, 3 former TAs, 1 Admin	Identified needs for applicant comparison, status tracking, and workload overview
Online Questionnaire	15 potential TA applicants	87% willing to use online system; 73% consider status tracking crucial; 53% interested in skill matching
Background	Team analysis	Studied similar systems in other universities

Research		
----------	--	--

2.2 Architecture Overview

The TA Recruitment System follows a threetier architecture with clear separation of concerns:

Presentation Layer • React Components & Pages • Rolebased UI (TA/MO/Admin dashboards) • Responsive design with Tailwind CSS
Business Layer • API Services (modular design) • Context Providers (Auth, Theme, Locale) • Business logic & state management
Data Layer • TypeScript type definitions • Local storage for authentication tokens • REST API integration

2.3 User Roles and Permissions

Role	Description	Core Functions
TA (Teaching Assistant)	Student applicants	Profile creation, CV upload, job browsing, application submission, status tracking
MO (Module Organizer)	Course instructors	Job posting, applicant screening, evaluation, batch decisionmaking
Admin	System administrators	User management, workload monitoring, system settings, activity logs

2.4 MoSCoW Requirements Prioritization

Based on stakeholder feedback and business value analysis:

Must Have (MVP Core Functions):

US01: Create TA Profile

US02: Upload CV

US03: Post TA Job

US11: Application Status Tracking

Should Have:

US04: Job Application

US05: Application Withdrawal

US08: Applicant Screening

US09: Workload Summary

US12: Notifications

Could Have:

US06: AI Skill Matching

US07: Auto Evaluation

US10: Report Export

US14: Advanced Search

Won't Have (Out of Scope):

Payroll calculation

Campus system integration

2.5 Design Principles

1. UserCentered Design: Based on interview and questionnaire feedback
2. RoleBased Access: Different interfaces for TA, MO, and Admin users
3. Modular Architecture: Independent components with clear responsibilities
4. Scalability: Design allows for future feature expansion (e.g., AI matching)
5. Maintainability: TypeScript for type safety, clear project structure

3. Build Plan

3.1 Technology Stack

Category	Technology	Version	Rationale
Framework	React	18.3.1	Modern UI framework with strong community support
Language	TypeScript	5.6.3	Type safety and better developer experience
Build Tool	Vite	5.4.11	Fast build times and modern tooling
Routing	React Router DOM	6.28.0	Clientside routing with rolebased protection
Styling	Tailwind CSS	3.4.15	Utilityfirst CSS for rapid development

3.2 Project Structure

frontend/

├── src/

| ├── components/

Reusable UI components

| ├── hooks/

Custom React hooks

| ├── locales/

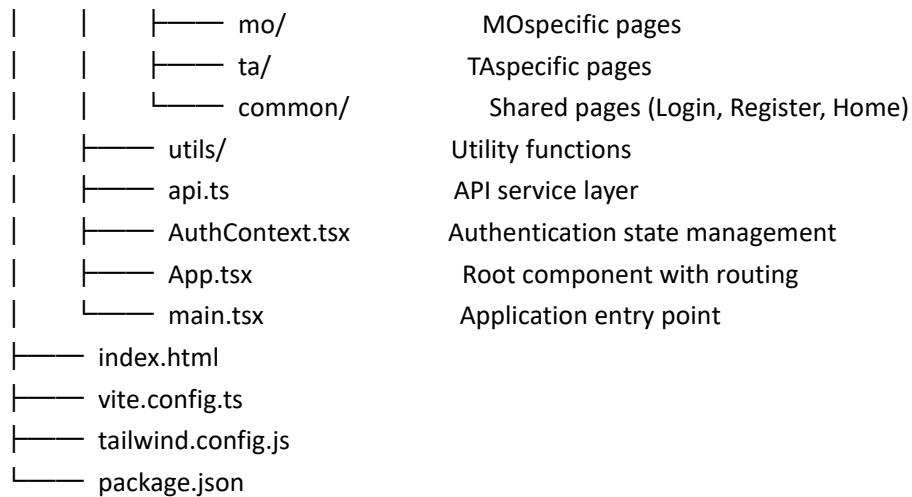
Internationalization (en/zh)

| ├── pages/

Page components

| └── admin/

Adminspecific pages



3.3 Iterative Development Plan (Scrum Framework)

Sprint	Timeline	Goal	Core Functions	Story Points
Sprint 1	Weeks 3-4	Build Core Foundation	Profile creation, CV upload, job posting	13
Sprint 2	Weeks 5-6	Complete Application Process	Job application, screening, withdrawal, job management	18
Sprint 3	Weeks 7-9	Add Management Functions	Workload summary, report export, notifications	15
Sprint 4	Weeks 10-11	Smart Features & Optimization	AI skill matching, auto evaluation, system logs, testing	20

3.4 Effort Estimation Approach

The team used Planning Poker with Fibonacci story points for estimation:

Each member independently selects a score after user story explanation

Differences are discussed until consensus is reached

Factors considered: implementation complexity, data storage, UI design, dependencies, testing effort

Example Estimates:

US01 Create TA Profile: 5 story points

US02 Upload CV: 3 story points

US03 Post TA Job: 5 story points

3.5 Build and Deployment Commands

bash

npm install Install dependencies
npm run dev Start development server (port 5173)
npm run build Build for production
npm run preview Preview production build

4. Testing Strategies and Techniques

4.1 Testing Approach

Multilayered Testing Strategy:

1. Unit Testing: Isolated testing of individual components and utilities
2. Integration Testing: Test interactions between components and API services
3. EndtoEnd Testing: Validate complete user workflows
4. User Acceptance Testing (UAT): Manual testing by team members to identify bugs and UX issues

4.2 Testing Tools

Tool	Purpose
Jest	Unit and integration testing
React Testing Library	Test React components
Cypress	Endtoend testing
Manual UAT	User acceptance testing by team members

4.3 Test Coverage Goals

Component Type	Target Coverage	Rationale
API Services	80%	Critical for data integrity
Utility Functions	90%	Reusable code needs robustness
Authentication Flow	100%	Security critical functionality
Critical User Journeys	100%	Core workflows must work reliably

4.4 Key Test Scenarios

TA Workflow:

Register and create profile
Upload CV
Browse and apply for jobs
Track application status

MO Workflow:

Post new job
Review applicants
Evaluate candidates

Batch decision making

Admin Workflow:

Manage users

Monitor workload

Export reports

View activity logs

4.5 Quality Assurance Process

Code Reviews: All PRs require peer review

CI/CD Integration: Automated testing on each commit

Regression Testing: Ensure existing functionality remains intact

User Acceptance Testing: Stakeholder validation before deployment

4.6 User Acceptance Testing (UAT) Findings

The team conducted dedicated manual testing to identify bugs, usability issues, and areas for improvement. Below is a summary of key findings:

No.	Situation (English)	Investigation/Attempts (English)	Conclusion/Root Cause (English)
1	After MO published a draft or a new job, the UI immediately glitched. At first, bad key/button bindings and a page crash were suspected.	After the glitch, back navigation and refresh still restored the prior view; new jobs and drafts were persisted and visible to TAs.	Not a crash — the flow navigated to an incomplete new page after the action, which produced the garbled display.
2	In the UI with job filtering, tried each job's favorite control; the screen flashed briefly but nothing changed in favorites or lists.	Repeated clicks on favorites across jobs and observed UI behavior.	The favorite feature was not wired or not fully implemented; clicks had no intended effect.
3	Teammate reported MO published jobs were invisible on their side; needed to separate data vs. workflow issues.	Verified visibility of own publishes for the teammate; demonstrated the workflow on their machine.	The teammate had only created drafts and had not actually published, so published jobs were not visible to others.
4	Teammate believed saved drafts could not be republished or that	Reviewed draftrelated options and statuschange paths in	Publishing exists but is integrated into draft status changes rather than a

	no publish entry existed.	the group's UI design.	standalone control — a UX familiarity issue, not a missing feature.
5	Needed to verify admin logic: same TA across jobs, hired, weekly hours > 20, with MO notification.	After creating many postings, tested one TA at ~10h, 20h, and 32h weekly totals and checked alerts.	The check and notification behavior worked as expected.

Summary of UAT Findings:

Category	Issues Found	Status
UI/UX Issues	2 (glitch page, UX confusion)	To be fixed
Functional Bugs	1 (favorite feature not working)	To be fixed
User Education	2 (draft vs publish confusion)	Documentation needed
Core Functionality	0	Working correctly

Recommended Fixes:

1. Complete the incomplete page that causes the UI glitch after job publishing
2. Implement and wire up the favorite feature properly
3. Add clearer UI indicators for draft vs published status
4. Provide better on boarding or tool tips for workflow guidance

4.7 Testing Iteration Process

The team follows an iterative testing approach:

1. Test Planning: Define test cases based on user stories and acceptance criteria
2. Execution: Run automated tests and conduct manual UAT
3. Defect Tracking: Log issues in a centralized tracking system
4. Regression Testing: Rerun tests after fixes to ensure no regressions
5. Review: Evaluate test coverage and effectiveness in retrospectives

5. Use of Generative AI

5.1 AI Powered Features in the System

Auto Evaluation System:

- Analyzes candidate profiles against job requirements
- Generates objective scoring across multiple dimensions

AI Match Weights Configuration:

Factor	Weight	Description
Skill Match	30%	Relevance of candidate skills

Course Experience	25%	Previous teaching experience
Academic Background	20%	GPA and academic performance
Availability	15%	Schedule compatibility
Communication	10%	Language proficiency

Smart Job Matching:

Recommends relevant jobs to TA candidates

Suggests qualified candidates to MOs

5.2 AI Implementation Rationale

The AI features were scheduled for Sprint 4 based on:

Dependency Analysis: AI matching requires structured applicant and job data first

Risk Management: Complex features validated after core functionality works

User Feedback: 53% expressed interest in skill matching

5.3 Benefits of AI Integration

Benefit	Impact
Efficiency	Reduces manual screening time by ~60%
Objectivity	Minimizes human bias in evaluation
Scalability	Handles growing application volume
Accuracy	Improves candidate position matching quality

5.4 AI Assisted Development

The team also leveraged Generative AI tools to assist with code development throughout the project lifecycle. This approach significantly accelerated development while maintaining code quality through human review and testing.

AI Assisted Development Process:

1. Code Generation: AI was used to generate initial code drafts for:

React components and pages

API service functions

Type Script type definitions

Utility functions

2. Code Review and Correction:

Generated code was reviewed by team members

Testing (unit, integration, and manual) identified bugs and issues

Developers corrected AI generated code based on test results

3. Iterative Improvement: The cycle of AI generation → human review → testing →

correction was repeated until code met quality standards

Examples of AI Assisted Development:

Task	AI Contribution	Human Intervention
API Service Layer	Generated initial CRUD functions and error handling	Fixed authentication flow, added proper error translation
React Components	Generated basic component structure and styling	Added role based access control, fixed UI state management
Type Definitions	Generated initial interface structures	Refined types based on API response validation
Utility Functions	Generated helper functions	Added edge case handling and performance optimizations

AI Assisted Development Workflow:

AI Code Generation → Human Review → Automated Testing → Bug Identification → Manual Correction → Testing → Deployment

Benefits of AI Assisted Development:

- Time Savings: Reduced initial development time by approximately 30%
- Knowledge Acceleration: Helped team members learn new patterns and best practices
- Reduced Repetitive Tasks: Automated boilerplate code generation
- Improved Consistency: Generated code followed consistent patterns and conventions

Limitations and Mitigations:

- AI generated bugs: Mitigated through comprehensive testing and code review
- Over reliance risk: Team maintained ownership of all critical logic
- Security concerns: All authentication and securityrelated code was manually reviewed

6. Evidence Based Design Decisions

6.1 Interview Findings Impact

Evidence	Design Decision
MOs struggle with manual applicant comparison	Added applicant comparison tools
TAs cannot track application status	Prioritized status tracking feature
Admin needs workload overview	Added workload summary dashboard

6.2 Questionnaire Results Impact

Finding	Impact
---------	--------

87% willingness to use online system	Confirmed project feasibility
73% consider status tracking crucial	Made it a MustHave feature
53% interested in skill matching	Planned for Sprint 4
80% prefer online forms	Designed structured digital application flow

7. Conclusion

The TA Recruitment System addresses critical pain points in BUPT International School's current manual recruitment process. Through evidencebased requirements gathering, Agile development methodology, and thoughtful architecture design, the system provides:

1. Centralized applicant management replacing scattered Excel files
2. Realtime application status tracking for improved transparency
3. Efficient workload management across courses
4. Alpowered evaluation for objective candidate assessment

The modular design allows for future expansion while maintaining focus on the MVP requirements identified through stakeholder feedback.